

Lec 06 Convolutional Neural Network - Part B

Lec 06 Convolutional Neural Network - Part B

Lecture	06 / 63 + intro
Duration	51:50
Captions	English (auto-generated)
Words	8,361
Video	wyvLZfmhqP0
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative AI and large language models. In the previous session, we had started with the CNN and then we have seen in detail working mechanism of convolutional layer which is mainly responsible for extracting the features. In today's session, we will explore the remaining components of CNN. We will look into the activation function RLU. Then we will see the pooling layer, flatten layer and the fully connected layer. Then towards the end we will understand how these layers work together in a deep CNN architecture. [snorts] So so far now if you see uh in the previous session we have understood that why images cannot be passed as an uh input to an

[00:50] multi-layer perceptron. The problem here is if you take image as a whole and when you convert that input dimension into a single vector the size is extremely large. Now when uh the same size when you pass it it leads to an spatial uh uh information is lost or the parameter explosion all these are the issues that you have seen. But you can ask uh from uh convolution layer we have still not reduced the dimension. In convolution layer it has mainly focused only on extracting the features but never you have reduced the uh size. If you use a padding whatever may be the size of an uh uh

image the same size is retained even in the feature map. So what exactly is an uh uh issue. Now we have to reduce the uh dimension of an

[01:44] image then only we can um you know pass it to an fully connected uh layers. Now in order to uh do it what all the different layers which will help us. So in the previous session we have only seen the convolution layer. Now we will be exploring the uh remaining layers. In that first let us look into the activation uh uh layer. So the activation layer here uh what we are using is a RLU rectified linear unit. What exactly this activation function is responsible for? It is responsible for all the pixels which are not uh contributing for um model prediction which does not contribute much the features which is being extracted from the convolution layer. If at all if there are any negative values which

[02:36] means those uh uh features are not contributing then uh RLU will eliminate those uh uh values. So that is the role of RLU which is an activation function. Now for example let us see this is the output of the convolution layer. So if I say output of convolution layer we would have taken an input we have taken a kernel. kernel is passed through the entire image and then we are getting the feature map. Now whatever is the feature map that we have obtained from the convolution layer, let us assume that feature map is whatever we are seeing here. Now when this is passed to an RLU activation function, what is ReLU does?

[03:22] ReLU is uh it takes the first value pass it to max of 0a 1. So what is max of 0a 1 maximum value is 1. So 1 is retained. Now 14 14 max of 0a 14 it is 14. Then you it takes the next number min - 9. So max of 0 comma - 9. So maximum between this is 0. So it is zero. Then four max of 0a 4 it is four. so on so forth. It will pass all the values which are been um all the values of the feature map is passed to an RU activation function and what we are going to see from this is you see wherever there are negative values all those negative values are converted to zero and all the positive values are retained. So what is the intuition behind this? The intuition is the this is the feature map. What is the

[04:22] feature map? It is the extracted features from the convolutional layer. Now all the features that it has extracted from the image. If at all if there are some features which are um not at all important which are negatively contributing then those features are not important for us. So rail activation function will eliminate all such negative values. So this is the intuition behind but if you see the feature map is still the same. Whatever is the size output size of uh activation function ru we have still not reduced the size of it. But ultimately when you have to pass it to an fully connected layer you have to start reducing rail layer will not help in reduction of dimension. It will only eliminate the

[05:14] features which are not important. Which features are not important? The val which are having negative values. So let's take this example. This is the same

example of our convolutional layer. There is an input image. that input image is passed to an kernel and from there we have obtained the feature map. Now when you look into suppose if an input image is X now uh and you have some kernel that you have taken this kernel has gone through the entire image and we have obtained this feature map.

[05:49] [snorts] Now if you look into this what exactly the feature how it has extracted the features. Uh observe this values very carefully. Can you see the diagonal values here? The diagonal values are very large comparing to any other values. The diagonal values are large. Which means what feature uh it has extracted when you have applied one kernel from the input image X. It has extracted this part. This is the feature it has extracted by applying one kernel. Now in this there are so many values which are not at all contributing. So when you apply it to RLU so uh this is the feature map obtained from a convolutional uh layer. So this when you pass it to an RLU activation function max of 0 comma 0.77 the maximum between

[06:43] these two is 0.77. Then it will take the next value max of 0 comma minus 0.11. The maximum between these two is 0. So we get zero and so on so forth. When you pass the entire values u [snorts] the entire feature map when it is passed to an RLU activation function this is the output of uh uh relu. Now if you observe here uh in this input image x if this kernel is responsible for extracting this feature. Now after the output of RLU observe this we have still retained the same we have nowhere uh compromised on that feature which has been extracted we are still having this only thing where you don't have any of the contribution from these uh positions all that are converted to zero this is what the RLU does. So uh in a

[07:42] short the uh role of a relu is it is uh an activation uh function which is going to eliminate all negatives to zero convert all zero negatives to zero. Now if you look into this is an input image to this input image you have applied this kernel and this is the uh feature map that we have obtained and when you pass it to uh rail activation this is the feature map that we have obtained. So uh this particular uh kernel has extracted this feature. So that is where you can see this entire value is high and the same values are retained here.

[08:22] So from this we have extracted this part. Now if you apply the same kernel once again the same kernel when you apply onto this input image we once again get the same values. We once again get the same feature map and the same um after we get this. So to extract a different uh uh features you take the same input but now you use a different kernel. We have seen this in the previous session right? We are using now a different kernel. This different kernel when it is passed through this entire image we are now going to get one more feature map. This one more feature map when it is passed to an RU activation function we are going to get this feature map. Right? And the similar way when we apply one more different

[09:11] kernel we get one more um feature map and when that is passed to an RLU we get one more output. I hope you can able to um visualize this. [snorts] This is what uh is the working mechanism of RLU. Now how does the output looks? You see this is an input image. when you apply it to the kernel uh this particular kernel focus on this when you apply onto this uh kernel and when you pass it to RLU we are going to get this output and if you observe it very carefully what is the most important it has retained only that in in the letter four what is what this particular kernel has extracted it has extracted all vertical uh points can you see this is vertical and here there is some portion and here there is some

[10:02] portion. This is what this particular kernel has extracted the features. Now to the same when you apply a different kernel you see now you have applied this kernel. So what this has extracted this has this has focused on extracting the vertical uh features. So it has extracted this part. uh previously it has extracted uh u this it has extracted this portion and now it has extracted this portion. So uh when you combine is it not when you combine this portion with this portion is it not we get something like four uh something like this we are going to obtain. So what is a relu in short? Before we move to the next one, it is going to eliminate all convert all negatives to zero.

[10:53] Right? Uh uh so how is that you write uh in the code? We have seen uh uh in order to perform a convolution operation, you require an input which is of size 64 641. You require number of uh uh kernels 32. Here number of kernels that we have taken are only two. But in general I mean in a real applications you'll be taking a larger number. Here we [snorts] have taken 32 and the size of each kernel how it is $3 + 3$. Size of such a 32 kernel uh size is $3 + 3$. Now uh the output of convolution has to be passed to an activation function. So the activation equal to relu is what is u been specified here. So this is what is relu. One important point you need to understand is convolutional layer is responsible for extracting the features.

[11:43] Rail layer is responsible for converting negatives to zero. But you have no way reduced the size. Whatever is the original input size. If padding is applied the feature map after relu is still the same as original input size. We have still not reduced any dimension. So [snorts] let us see how we will reduce uh in a later slides. Now let us understand how many number of parameters are there uh when to perform this particular operation. What operation convolution operation then you have um you know passed it to RLU. So to uh to perform this entire how many number of parameters are uh there how many uh total parameters that that are required in performing this particular operation. So what is that you require?

[12:37] You require an input here the input is $32 \times 32 \times 3$ here is RGB color image that we have considered. Size of the filter that we have considered is 5×5 . Now why it is 5×5 ? Why not 3×3 ? It's an hyperparameter as I said. So for this particular example, we have considered 5×5 . How many such filters we have considered? Six such a

filters we have considered. Now stride doesn't help us with number of parameters. So any value it does not affect the parameter count.

[13:12] Padding whether you apply padding or you don't apply padding. So once again it does not affect because for original $32 + 32$ if you apply padding but original input is still $32 + 32$ right that is the reason padding even if you do padding or don't do padding for calculating the number of parameters it doesn't have an impact so what is the formula that is used for calculating this the formula is $um \times fh$ into fw that is a kernel or a filter um height and width here it is $5 + 5$. Okay. Then number of channels how many such channels are there? So number of channels you will get from the input.

[13:57] If you are considering an RGB number of channels here are three. If you are considering an grayscale or binary the number of channels will be one. So since it is an RGB the channel is three plus one bias term and um into how many such um kernels we are considering. number of such kernels or filters are six. So apply um substitute the values in this particular formula and then when you calculate we are going to get this. So wherever you come across the calculation um you can just calculate and see if there are any typos you can feel free to reach out. So here it is 456 um parameters.

[14:43] So which means to perform a convolution operation we are um um for this particular um um input um size we are having totally 456 um learnable parameters or parameter these many number of parameters are there. So if you have observed what exactly we are learning, we are learning the values which are there in this and this $5 + 5$ because as I said in the previous session um this kernel or a filter is a learnable parameter the the values which you are going to use are not a fixed values over a period of time in the iterations the values are going to get updated. How many such values? It is 456 parameters.

[15:30] Similar way let's take um one more um example here. Filter width and height is $3 + 3$. So which means you will have $3 + 3$ and um number of such um channels here are three + 1 and then how many such filters? Number of such filters are 64. So when you do this um um calculation we are going to get this output right. Similar way for the next convolution the number of filter um the filter size is 5 cross five number of such filters what we have considered 64 and + 1 and um input channel is 64 and number of such kernels what we have considered is 128. So when you calculate we are going to get this. So um just see these values and you can correct um yourself if there is any typos. So this is how you calculate how many um

[16:30] learnable um um parameters. This is important concept. You should always know how many such a learnable parameters are there after every layer the um after every operation that is performed. Okay. Now um this is important slide. What exactly is this is? Now you have passed an input image. That input image is passed through a convolution layer and um the output of convolution layer is a feature map. Right?

That feature map is also called as an output of that convolution layer. Right? Now what exactly will be the size of uh the uh convolution layer? Output of convolution layer. When an image pass through a convolution layer, what will be the output size? And the formula is uh uh given as $I - F + 2P$ divided by S

[17:23] which is a $str + 1$ into D . Now what exactly is this? Let us uh take with an example and then understand. Now let us assume this is the example where you have an image of size $32 \times 32 \times 3$ when I say 32×32 it's a dimension. When I say three it is an RGB image. filter is a 5×5 which means 5×5 filter and the number of filters are six. So which means you have an input image which is of size 32×32 and there are three such channels because it's an RGB three such channels and there are six filters of size 5×5 there are six such filters there are six such filters now stride that you are considering is one and there is no padding that is applied now if this is the case after to perform what will be the output size. Now you

[18:24] know very well that when one kernel is applied on one you are going to get one feature map when a second kernel is applied on an input image you get another feature map. So now how what will be the size at the output. So just substitute the values in this. Here I stands for input which is 32×32 . F stands for filter which is 5×5 and $2P$ stands for whether there is a padding or no padding. So since there is no padding this entire term becomes zero. Then stride since a stride is one denominator is one. And then we have depth. D stands for um you know number of such filters.

[19:08] So output channel which is a filter. So look at this when we uh uh calculate what is uh that we are going to get I value is 32 filter f is five uh uh padding there is no padding so zero divided by stride is $1 + 1$. So $32 - 5 + 1$ we are going to get 28. If you have observed here there is no padding that is applied. When there is no padding then obviously the uh feature map size is going to reduce that is where input size was 32×32 after you performed it is 28×28 . Now you can ask what is this D . Now just imagine when the first kernel is applied onto this input image we get one feature map. When second kernel is applied onto this input image we get second feature map. like this. How many such a feature maps will

[20:04] be obtained? It definitely depends on how many kernels that we have considered. Right? So since here number of kernels that we have considered um is six. The output you will have six such 28×28 feature maps. So this is important. Why it is important? tomorrow the when uh you try to propose your own uh CNN architecture or when you come out with a custom CNN architecture when you are when you are writing uh an architectural diagram always we should specify after we perform one convolution layer what is the output uh size after we perform another convolution layer what is an output size so that um you are saying for your architecture diagram what is the input past and how each of the layer is giving you the output. Now

[20:55] if you doesn't want to get into this math but if you want to understand then in

the hands-on you will see something called model dot summary. When you give model dot summary after every layer what will be the uh output you'll be able to visualize that. Now let us take the same input but where now you are applying padding equal to two. When I say padding equal to two, it is for this input image, it is one layer and it is one more layer. That's the meaning of padding. There are two layers of padding that is applied. That is what padding equal to two. So what will be definitely when you apply padding always it will retain uh the uh size of an original input.

[21:42] Original input size is $32 + 32$. apply the formula. So 32 uh and the filter size is five and padding is 2 P. So 2 into P value is 2. So it is 4 stride is $1 + 1$. So when you calculate we get the value 32. Right? Now when you when you look into this whatever why why we are getting the same uh output as uh same size as input because padding is been uh has been applied. So this is how you will calculate uh the size uh of an convolutional layer that is a feature map output uh size. [snorts] Now uh you can ask what if there are some calculations where we are going to get a decimal value then it is very simple go to floor uh apply floor you're going to get the value as an integer value only you will not get a decimal. Yeah. So

[22:42] next so so far what we have observed is you have performed a convolution operation we have seen an RLU activation function in both of these two we have nowhere reduced the uh dimension of an u feature map. Whatever is the original input if the padding is applied we are retaining the same uh uh feature map. But when you want to apply it to an fully connected you cannot pass the same because uh um you know parameter exploration is going to happen. So definitely we have to reduce the size.

[23:19] Now if you have to reduce the size what is that operation that you will do so that now you are going to retain only the most important features which are sufficient to say it is so and so. Okay. Now for example you take uh um you know some uh you want to identify some uh human being somebody if you want to identify a person you don't require the entire features of a person to identify some important features are sufficient in order to say that uh the person uh uh name in order to identify a person [snorts] pooling does that uh pooling now going to reduce the size of an uh feature map. Now when it is reducing the size of the feature map, it has to consider some important aspects. Now let us understand what exactly is pooling a

[24:09] layer. How does the pooling a layer works? It requires the output of relu activation. Here this is what is the output of rail activation where all negatives are converted to zero positives are retained as it is. Now I cannot pass the same to a fully connected because the size is large. For this particular example, for understanding, we have taken a smaller image. But in a real world, you will not have an image of size $7 + 7$ or $8 + 8$.

[24:37] You will not have such a small dimension one. [snorts] So this is the output

of rail activation. How pooling works is it requires one fixed size window. So uh this is the window that we have fixed here. We have considered it as 2×2 . Now once this fixed size is considered now you have to perform an summary operation. What is summary operation here is now uh in this window how many values we are seeing? We are seeing totally four values. What are those? 7 7 0 0 and 1. Now the summary operation is whether you want to consider the maximum highest value or whether you want to consider the lowest value or whether you want to consider the average of all. So that is what is the summary operation.

[25:26] First uh fix um window that fixed size window here in this example we are considering 2×2 and this window will slide over the entire feature map. Okay. Then we have an summary operation. What the summary operation is? Once you fix the window in that particular slot, you apply an operation called max pooling or you can apply an average pooling. We can also consider min pooling. I'll tell you that now. For example, if you apply max pooling, you're going to get the value one. Then uh um you are going to stride it and then you perform remaining that you'll see in the next slide. So by doing this is it not you're going to get one output you are going to get this output and if you see the dimension has

[26:16] definitely reduced from this uh size you have reduced it to 4×4 so by doing this what exactly we have achieved we have uh done the down sampling therefore pulling in CNN is a downsampling technique and what it is going to do it is going to reduce the dimension spatial dimension that is width and height of what of this feature map and why is that we are doing this definitely it will reduce the computational cost and that is what is required to pass it to an fully connected layer so from this you are only understanding what is a pooling it is a downsampling technique how it works it requires a fixed size window which will slide over the entire and when a window is on a particular part of a feature map you will apply a summary

[27:07] operation. Those summary operations can be any one of these. For example, let us understand the types of pooling here. Let us uh take an example where the feature map size is 4×4 . Now, uh you have taken an window size as 2×2 . So, window size is 2×2 which means you will take this particular window. Right? Now when you take a max pooling what is a max pooling? When a window is placed on the onto this feature map whatever the portion that we are visualizing in that what is the maximum value. So the maximum value is 71. So 71 what is the meaning of this is the intuition behind is the most important feature is present here that 71 carries the most important uh feature. Yes. then you have to stride it. So here the

[28:06] stride will be whatever is the window size by default. Otherwise explicitly we can specify the stride also. If you don't specify whatever is the size of the window that will be the stride uh uh value since window size is 2×2 stride value is two which means you will now uh uh two you will leave these two and this is the next window size and the max pooling maximum among this entire is six. So six value then again

stride is two. So you will leave these two and then you will come to this portion. So maximum is 60. Then again two you will leave and this portion. So here maximum is 13. So this is what is uh pooling where you take a fixed window that will stride over the entire when a window is placed it is going to perform

[28:57] a summary operation. Second is average pooling. What is average pooling? It is um uh $71 + 8 + 1$ is and plus 0 is totally 80. 80 divided by 4. Why four? Because four such values we are considering. 80 divided by 4 is 20. Similar way you will add all these four values and you will divide it with four. You get this. You will add all these four values divide it with four you get this. Similar way with this. What is the difference between average pooling and max pooling is in max pooling you are going to take the maximum value. In average pooling you are considering the maximum the uh important feature that is max value and you're also considering the minimum value and then you will take the average of it. So you get this

[29:50] average pooling. Then we have minimum pooling. you are um focusing only on the least uh important features. So the minimum value here is zero two here 20 and here it is three. This is a minimum value. Recently I've come across one more pooling which is um max average min pooling. How this uh pooling works is look into this. This is the uh mathematical explanation for um max average min pooling. how you are doing. We will first calculate what is max minus uh minimum. So now let's go back and then understand what is um maximum in this entire value. Maximum value is 71. And what is the minimum value?

[30:38] Minimum value is zero. So max minus minimum is 71. And if that value is greater than average, what is average here? It is um okay let's go back. Average is 20. So 20. How do we get average? Add all the values divided by four. Which values? Only that window size values. So here 71 is definitely greater than average. Then if that is the case, what is the output? Output is max minus minimum which is 71. Then let us uh consider uh suppose if this value if this condition does not satisfy if a max minus minimum is less than or equal to average then what value you are going to consider is max plus average divided by two. So this is max average minimum uh uh pooling this is one more pooling which you can uh look into when you are

[31:36] doing an implementation. So uh these are four different and if you understand the intuition when the difference max minus minimum when this difference is maximum um and when the difference between maximum and minimum value is large that means when this difference value is very large which is 71 in our case in the first example which means that is the window which has a strong features or um there is a sharp change there. So in that case we need to retain that sharp information. How do you retain? You will retain that value. Suppose if a max minus minimum is very less or moderate which means the information uh the window is more uniform uh uh in that case what is required to you is you require this uh uh information max plus

[32:32] average by two. You're going to have this information. So this is one of the um uh pooling which have come across in one of the research paper. So uh I thought we can discuss that here. So max average minimum uh pooling. Then let's go back to the same example uh where we had stopped. So we have a convolution layer which has extracted the features that is passed to a rail and then uh this is the output of RLU activation function. Now for this you are applying a pooling and we know that pooling will always reduce the dimension of an image.

[33:08] Now let us see how it is going to reduce it. What is the size of uh this particular it is a 7×7 uh cross 7. So this is 7×7 feature map that is applied to an max pooling. For max pooling you require a window size as a fixed window size. It's an hyperparameter. Here in this example we have taken 2×2 and the uh stride if you doesn't specify anything the stride is two. So now when you perform here we are applying max pooling. So among this entire the max value is one. So one. So since stride is two we are leaving the first two and then take the next window.

[33:51] So here the maximum you are applying it to max pooling. Maximum is 0.33. Again you leave a two and then a stride which will be this value. Again you will take the maximum which is 0.55. Now leaving a two when you take a stride. So here you just have only two values remaining will be 0 0. So among this the maximum value will be 0.33. Now stride when you take you leave the first two rows and you will now consider this part of an um feature map is so on so forth you will get this value. Now you can ask the question by doing a down sampling is it that we have lost some information.

[34:34] The answer is no. Observe this very carefully. uh uh when you look into the diagonal this is what is the diagonal values which is uh the original input x which has extracted this part. Now after you apply a max pooling can you observe this diagonal you still have that. So uh by reducing the dimension the most important features are still retained. We are not compromising on just by reducing the dimension. We are not compromising on the important features that it has extracted those important features are retained. So this is what is uh max uh pooling. So how do you write when you are doing uh an hands-on implementation when you want to implement this? What is that max pooling 2D? We are implementing it in KAS. In

[35:28] Keras you have an function called max pooling 2D. Uh why max pooling? because we want to perform max pooling. Now uh in this the compulsory parameter is you have to fix the uh window size that pool size window size is 2×2 when you write 2×2 by default the stride here is two default otherwise you can explicitly specify as well when we are doing a mathematical I mean when you are implementing it see how we are going to implement it first you require an convo 2D because you have to perform a convolution operation you require an input which 28×28 you require an RLU activation function. We require padding equal to same which means we are applying a padding now and we have considered 32 such filters and then $3 \times$

[36:19] 3 uh size of a filter. Uh so once you do this you are going to get an output and uh the output will be once again $28 + 28$ but what will be the um how many such kernels what will be the depth here the depth will be number of such filters number of such filters here are 32. So this will be the output after we perform the first uh line when you pass it to max pooling. So this is $28 + 28$. So pooling you are taking a window size as $2 + 2$ and stride if you don't specify by default it is uh two. So uh when you u perform an max pooling you are going to reduce the size to $14 + 14$. Why it is a two. So definitely it will down sample exactly by half and uh um you you you will not reduce this because 32 such

[37:11] feature maps has reduced to a size $14 + 14$ which means this 32 will be retained as it is right. So padding equal to same. Why? Because in convolution you have applied a padding. So even in max pooling you will be applying a padding. That is the reason why uh when you don't had a values there it is 0 0. So this is the um uh this is how you will write m uh you know in a programming way the operation since these two are the functions which are part of um kas. So you will first implement uh you will import the keras from tensorflow and you are building a models and the that model you're going to build a sequential model one after the other uh you are going to place a layer so you will import sequential from where from

[38:02] keras from tensorflow docaras models then you have to import these two standard uh functions standard methods so you will import convert 2D and max 2D these are the layers. So from tensorflow.keras.layers you will import these two. However towards the end of this particular uh uh week you will see all these in the hands-on. So I hope pooling is clear. So stride will be by default uh the window size. Uh uh you if you want you can explicitly specify the stride then you have to write it in the program stride equal to $1 + 1$. If you don't specify it will take window size. If you uh want to change it, you can uh write explicitly.

[38:50] Uh so this is what is the pooling layer. Now let us understand how do you calculate the output size. Now here how do we calculate this size? Um what will be the feature map size after the pooling layer. So after the pooling layer the formula is this. It is $NH \text{ minus } F \text{ by } S + 1$. uh this is height to width into uh depth number of filters that are used in the convolution layer. Let's directly take this with an example. Suppose if an input image is $5 + 5$ and um here the channel is one which means the depth here is one and um number of such filters what we are considering here are three and the stride here is one. So just substitute the uh values I'm I'm using it for the height same thing into uh it will be for

[39:45] the width. So it is uh input so $nh \text{ minus } uh \text{ } f$ so it is $5 \text{ minus } 3$. So uh and divided by $1 + 1$ so we are getting an output as three and this is height is three width is three. So this is what and into depth. Depth here is since the number of uh such filters you have considered is one. So depth here is one. I hope this is clear to you. So uh this is what is the um so $5 + 5$ this is $5 + 5$ uh filters um are three such a filters

stride is one. So what will be the what is the output that you have obtained?

[40:28] The output that we have obtained is 3×3 . And don't consider this is one output. This is second, four, 3×3 , four, five. No, absolutely no. You have taken the window and you are moving the window throughout uh the program. So finally it is only one uh 3×3 into 1. This is what is the uh pooling uh layer. Um I hope uh this is uh clear. So this is a important concept uh where we are summarizing the entire you have taken an uh input image for that input image you have passed to an convolution rail and pooling and you have taken three kernels. So how many feature maps we have to obtain three. So those are these three feature maps and I hope you have understood this. This kernel is passed through the image you get this

[41:19] feature map. Second kernel is passed to uh to the image and um second kernel is um you're passing it to the entire image. The kernel is going through the entire image and you get the second feature map. Similar way the third feature map to do this how are we writing a code? Input size here is $7 \times 7 \times 1$ and then padding equal to same. That is the reason you are seeing that there is a padding layer that is applied. railway activation uh um 3×3 filter and three such filters have been applied and um after for this pooling this is the uh line of code. [snorts] Now you can ask uh uh can we repeat this entire process? uh why we have to repeat it what if you want to um you know perform once again convolution rloop pooling is

[42:10] it possible answer is yes you can definitely do one more I'll tell you why to do one more in the upcoming slides so let us assume now you are again performing one more convolution ru and pooling layer and you have used three kernels what will be the output now so this kernel will be applied onto this uh entire three matrix X we get one feature map. The second kernel onto this entire you get one more second third kernel onto this we get one more feature map.

[42:42] So since there are three kernels so definitely you will have the three feature maps but definitely the size will be reduced. Why size will be reduced? Because now the input is not 7×7 . The input is uh these matrix. This is what you are passing as an input to the next set of uh uh operation. So this is the feature map uh that we will obtain. We obtain this feature map. Now you can ask the question why why to do this. So let us uh uh I hope this is clear to you. You have taken an input image three kernels you passed it to entire convolution. We got these feature maps.

[43:19] Again we are performing an other set. So we are again taking the kernels. So for the second convolution you see the way that we are writing we are now applying the con 2D uh for this we are taking three kernels and each kernel of size 3×3 since it is a sequential model no need to write explicitly the input definitely the previous output will be the input because we are building a sequential model that is the reason for second time con 2D we are nowhere passing anything about the input then relu is uh written here and then the pooling pooling has been specified we get this output.

Now uh uh let us understand why to do that. Before that we will see this topic called flatten layer. What is a flatten here

[44:06] ultimately? What is that we require? Now there is an image you have extracted the features negative values are converted to zero. Most important features you have retained by doing pooling. So but we have still not classified anything the image that whatever the features that you have extracted what is that image whether it's a cat dog what exactly is that image nowhere we have classified still for classification you have to pass it to fully connected layer now go back from our previous session in order to pass it to a fully connected layer our im so far our data is in the matrix form right it is in a two-dimensional matrix form now uh that has to be converted into a single vector. Why it has to be converted?

[44:51] Because multi-layer perceptron expects the data in the in a single vector. So therefore, what exactly is the flatten layer doing? Whatever is the output. So this is the output whatever is the output that we have obtained after pooling a layer. Whether you do one pooling, two pooling, multiple poolings, whatever the output of pooling layer, so that will be in the form of a matrix. So that matrix now should be converted into a single vector. How is that you do that into a single vector? Can you see this?

[45:23] We have a feature map. This feature map which is in the matrix form is converted into an single vector. How is that you are doing? So it is 31 1 3 1 2 5 0 250 then 42 142. So you are converting it into a flatten and then you can pass it. So what if you have multiple such a feature maps here there is only one feature map you have converted when you have multiple such a feature maps you can do this uh this will lead to one flatten this is flatten and this is flatten so how do you perform this operation you perform this just by calling a flatten function why do we require this because ultimately our aim of using CNN is either classification or you can do a segmentation Or you can do an object detection. Here what we are

[46:17] doing is the classification. If you are using a classification, you have extracted the features, you have reduced the dimension, but you have still not classified what exactly it is. To do that, you have to pass it to fully connected layer. To pass it to fully connected layer, that requires a data in one dimension in one vector form. To get that in one vector form, the feature maps which are in matrix are converted into a single vector. uh that is a flatten that is what it is doing. Now let us try to understand why did we repeated it once again. Now let us assume this entire portion is not there.

[46:52] We have taken an input image. We have passed it to convolution rail loop pooling and then we have obtained this uh uh output. This is our feature maps right feature maps. Now uh let us assume after this we are passing it to a fully connected layer. If you have to convert this into a single vector, can you see what is the uh dimension? Uh here it is $4 + 4$ which is 16. Here you have 16. Here we have 16

which uh I'm sorry which means 48 are the features that we are passing to an um fully connected 48 here. This is a very simple example which is of 7×7 . But can you imagine this for a real world application where your image size is of 256×256 and after you perform one set of convolution what will be the dimension?

[47:47] So then if you pass then you convert that into a single vector again is it not a larger value? I hope you are getting this right. So you cannot directly pass uh such a huge dimension into an um you know flattened layer uh convert that into flattened layer and pass it to fully connected. So therefore this feature map size is still large. In order to reduce that we are again passing to an next set of convolution. By doing this we have got a less uh uh dimension feature map. Now what is this size? Four. This is a four. This is a four. So is it not just 12 feature 12 features that you are passing it to and fully connected? I hope you are getting this. So that is the reason why we have repeated it once again. Now uh from uh

[48:39] smaller 7×7 for uh three uh such kernels 3×3 size we have got 48. Again performing you have got 12. So with this can you imagine if the uh dimension is 256×256 how many such a times you should repeat so that the final feature map size will be smaller so that you can pass it to an fully connected I think you should be able to get that intuition now right so once you convert that into an flattened layer the next is your fully connected layer so this is your multi-layer perception so you can have any number of hidden layers. You can have any number of neurons in the hidden layer.

[49:22] And the last layer depends on how many number of classes we have. If you are doing a binary classification at the output layer, you will just have one neuron. If you have a multiclass classification, number of classes is what will be there in the output layer. If you are doing a threeclass classification, you will have three neurons at an output layer. It's a classification problem. So either it is a binary or it is a multiclass. So how is that? You will be writing a code. You will first call a flatten layer so that the feature maps which are in matrix form will be converted into a single vector. Then 128 is the um first um you know it is an hidden layer which is having 128 neurons. Activation function

[50:09] used is ru. Then uh I'm not writing the code for this diagram. It is uh the the code that is mentioned here is a general one. Then since there are three neurons here at an output there is a three and the activation function that we are using is soft max. From our previous uh activation session, you must be knowing if it is a binary classification, number of neurons you will write here as one and activation you will use sigmoid because if it is a threecl class uh problem, number of neurons will be number of classes and the activation function is soft max. After that you will compile the code. To compile the code um you will be using an optimizer and you'll be using a loss function. And what is the metric that you are

[50:54] calculating? It's an accuracy. However, this you will see in the hands-on. The whole intention of uh uh the last two sessions uh that is the previous session and this session is to give you the overview of how CNN works. So we have a convolution layer responsible for extracting the features. We have an activation function responsible for uh removing all negatives to zero. Then we have a pooling layer which is used to downsample it. And then we have an flatten layer which is used to convert the feature maps into a single vector and then pass it to an fully connected layer. So with this we have come to an end of this particular uh session and we have come to an end of theory session of week one. In the next two sessions

[51:41] you'll be seeing an hands-on on this CNN. Thank you.